

Scan2BIM Segmentation Microservice Architecture and Processing Flow

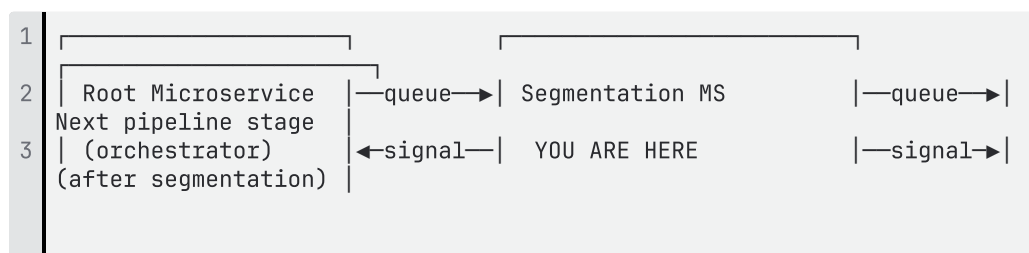
The Segmentation Microservice is the core machine-learning stage in the Scan2BIM pipeline. It receives a segmentation job from the Root Microservice, downloads the source E57 point cloud, performs GPU-based semantic segmentation, derives wall geometry to refine the scene bounds, and then hands structured outputs forward for downstream processing. This page describes the service **before the Post-Processing Microservice**, with emphasis on architecture, runtime flow, data movement, and key behaviors.

Pipeline position: Root Microservice sends work to Segmentation through the segmentation queue. Segmentation produces inference outputs, extracted images, and wall geometry, then signals workflow status back upstream and dispatches artifacts downstream for later post-processing.

- [Architecture Context](#)
- [Service Responsibilities](#)
- [Component View](#)
- [End-to-End Processing Flow](#)
- [Two-Pass Segmentation Logic](#)
- [Detailed Functional Stages](#)
 - [Message consumption and orchestration](#)
 - [Image extraction](#)
 - [Wall extraction and geometric refinement](#)
 - [Inference execution](#)
- [Data Inputs and Outputs](#)
- [Integration Boundaries](#)
 - [Upstream: Root Microservice](#)
 - [Downstream: next-stage processing](#)
- [Failure Handling and Operational Behavior](#)
- [Key Takeaways](#)

Architecture Context

The service operates in a queue-driven, job-style execution model. Instead of behaving like a long-running request/response API, it consumes a work item, processes the scan, persists outputs, sends completion signals, and exits when no more messages are available. This makes it suitable for burst-based scaling with GPU-backed workers.



Area	Details
Primary role	Run semantic segmentation inference on E57 point clouds and prepare outputs for the next processing stage
Execution model	Job mode; processes queued work and exits when the queue is empty
Implementation	Python-based service using Pointcept on PyTorch with GPU acceleration
Health endpoint	Port 5000
Scaling model	KEDA-driven autoscaling based on queue depth, typically scaling from zero to active replicas
Infrastructure profile	GPU-enabled runtime with high memory allocation for large point-cloud inference

Service Responsibilities

Before any downstream post-processing begins, the Segmentation Microservice is responsible for the full transformation of raw scan input into segmented, structured intermediate outputs.

- Consume a segmentation task from the queue
- Parse and validate the incoming message envelope and payload
- Check job state and avoid duplicate processing
- Download the source E57 file from Blob Storage using the provided SAS URL
- Extract panoramic or embedded scan images in parallel
- Run pass-1 segmentation on a broadly cropped version of the scan
- Detect walls and compute wall-bounded geometry from pass-1 results
- Run pass-2 segmentation on a tighter, refined crop using wall-derived bounds
- Upload segmented outputs, wall metadata, and images to Blob Storage

- Signal completion or failure back to the Root Microservice
- Dispatch a prepared payload for the next pipeline stage

 **Important scope note:** this page stops at the handoff point. It focuses on what Segmentation produces and how it behaves **before** the downstream Post-Processing Microservice starts its own work.

Component View

The codebase is organized around queue orchestration, segmentation execution, geometry derivation, storage helpers, workflow signaling, and operational monitoring.

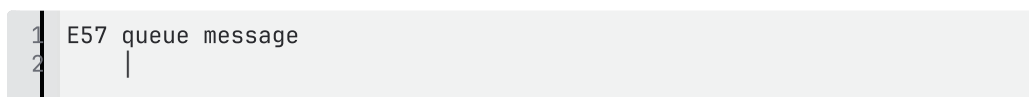
Module or Area	Purpose in the service
<code>main.py</code>	Application entry point; starts the queue consumer in job mode
<code>queue/message_consumer.py</code>	Consumes Service Bus messages and orchestrates end-to-end processing
<code>queue/message_producer.py</code>	Sends completion payloads and downstream queue messages
<code>service/segmentation_service.py</code>	Wraps the segmentation pipeline and exposes service-level execution logic
<code>service/wall_extraction_service.py</code>	Detects wall planes and computes bounding geometry for refinement
<code>segmentation_pipeline_mp.py</code>	Implements the two-pass segmentation pipeline and multiprocessing flow
<code>cropping_algo.py</code>	Performs XY cropping, histogram filtering, and bounding-box/rotation-based crop logic
<code>inference_ppt.py</code>	Runs Pointcept model inference on the cropped point cloud
<code>image_extractor.py</code>	Extracts images from E57 inputs in parallel
<code>database/jobs_repo.py</code>	Tracks processing state and supports deduplication through MongoDB
<code>schemas/message_schema.py</code>	Defines structured queue payload models

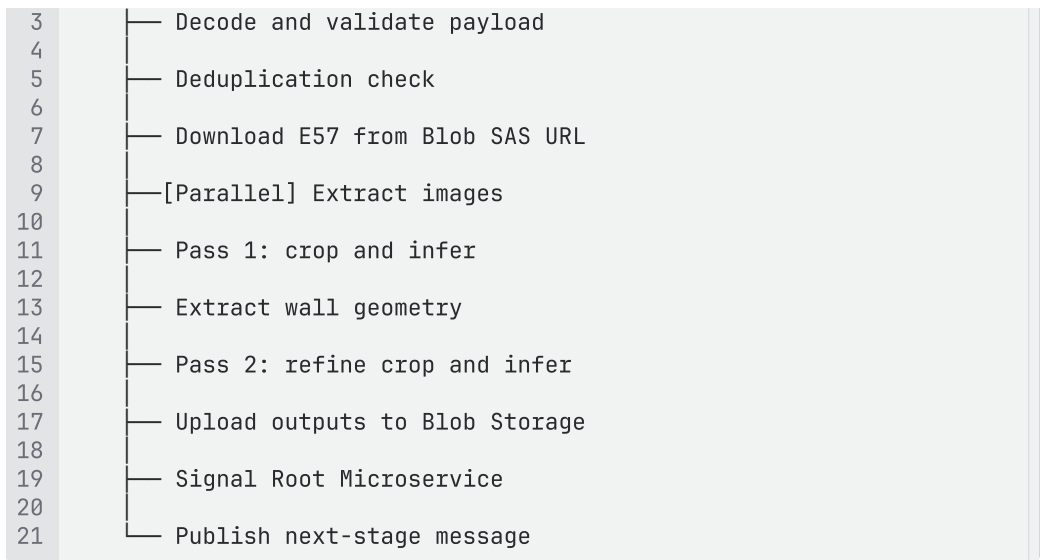
<code>temporal/client.py</code>	Sends progress or completion signals into workflow orchestration
<code>helper/*</code>	Provides Blob, authentication, and structured logging utilities
<code>monitor/*</code>	Tracks resource utilization such as memory and GPU behavior
<code>wall/*</code>	Contains geometry and wall-processing helpers

End-to-End Processing Flow

The microservice follows a predictable sequence from message intake through artifact upload and workflow signaling.

1. **Message retrieval:** a worker reads a message from the segmentation queue.
2. **Envelope parsing:** the service unescapes and decodes the outer queue message, then parses the nested segmentation payload.
3. **Validation and deduplication:** identifiers such as workflow ID, task ID, and trace ID are validated and checked against persisted job state.
4. **Source acquisition:** the E57 input file is downloaded using the SAS URL contained in the payload.
5. **Parallel image extraction:** embedded images are extracted in parallel while segmentation preparation begins.
6. **Pass 1 inference:** a coarse crop of the scan is segmented using the GPU model.
7. **Wall extraction:** wall planes are identified from pass-1 outputs and used to derive bounding geometry, elevations, and rotation metadata.
8. **Pass 2 inference:** a refined crop is generated from wall-derived limits and segmented again for better precision.
9. **Output persistence:** inference artifacts, images, and wall metadata are uploaded to Blob Storage.
10. **Workflow signaling:** completion or failure is reported back to the Root Microservice.
11. **Downstream dispatch:** the next queue message is published with references to segmentation outputs.





Two-Pass Segmentation Logic

The defining characteristic of this service is its **two-pass segmentation design**. Instead of relying on a single inference run over the entire scan, it first performs a broad segmentation to understand the scene, then uses detected walls to constrain and improve the second pass.

Pass 1: coarse segmentation

Runs on the broadly cropped scan after XY-based filtering. Its goal is to identify semantic classes and provide enough structure for wall detection.

Pass 2: refined segmentation

Runs on a tighter crop defined by wall boundaries, elevations, and rotation metadata. Its goal is improved precision within the shelter footprint.

Stage	What happens	Why it matters
XY crop	Histogram-based filtering removes peripheral noise and prepares a manageable point-cloud subset	Reduces irrelevant geometry before inference
Pass-1 inference	Pointcept segments the point cloud into classes such as wall, door, rack, cabinet, battery, rectifier, floor, and ceiling	Creates the first semantic understanding of the scene

Wall extraction	Wall planes are detected and converted into bounding boxes, elevation ranges, and rotation information	Provides a geometry-driven boundary for refinement
Refined crop	The point cloud is cropped again using wall-derived bounds; if rotation is present, the cloud is aligned before cropping	Improves focus on the meaningful interior region
Pass-2 inference	The same model is rerun on the refined crop, with inverse rotation applied when needed	Produces a cleaner and more precise segmented output

i Why two passes help: the first pass provides structural cues, while the second pass removes much of the surrounding noise and boundary ambiguity. This is especially useful when the shelter is rotated relative to the scan axis or when the raw scene contains clutter beyond the target structure.

Detailed Functional Stages

Message consumption and orchestration

The queue consumer is responsible for turning a transport message into a fully tracked processing job. It handles decode steps, payload parsing, deduplication checks, storage download, pipeline invocation, upload, signaling, and dispatch.

The service effectively bridges workflow orchestration and GPU inference. It is both a message-driven worker and a domain-specific processing engine.

Image extraction

Image extraction is started in parallel so that visual assets are available without delaying the main segmentation flow more than necessary. These images are later uploaded alongside segmentation artifacts for use by downstream services or review workflows.

Wall extraction and geometric refinement

Wall extraction is central to the second pass. From the first-pass semantic outputs, the service estimates wall planes and derives:

- **2D wall bounding box** for plan-level crop control
- **Elevation limits** for vertical crop constraints
- **Rotation metadata** for scans where the shelter footprint is not axis-aligned

If the structure is rotated, the pipeline temporarily rotates the point cloud into an aligned frame, applies the crop, runs inference, and then transforms the output back into the original orientation.

Inference execution

The service uses GPU-backed Pointcept inference with memory-conscious settings suitable for large point clouds. Streaming and memmap-based accumulation reduce memory pressure while preserving throughput on large scans.

Inference setting	Value or behavior
Framework	Pointcept on PyTorch
Device	CUDA GPU
Method	Block-stream inference
Block size	10.0 meter processing windows
Accumulator	Memmap-backed accumulation for large inputs
Precision	Float16 accumulation support for efficiency

Data Inputs and Outputs

The microservice transforms a single queued input into a set of persisted intermediate artifacts. These outputs are the contract boundary before downstream post-processing begins.

Category	Produced artifact	Purpose
Input	E57 scan file from SAS URL	Source point cloud for segmentation

Intermediate	Cropped point clouds for pass 1 and pass 2	Prepared subsets used during inference
Inference output	Segmented PLY files from pass 1	Initial semantic segmentation results
Inference output	Segmented PLY files from pass 2	Refined semantic segmentation results
Geometry output	walls.json and related wall bounds	Stores wall planes, bounding boxes, elevations, and rotation context
Visual output	Extracted image files	Preserves imagery associated with the scan
Workflow output	Completion signal payload	Updates orchestration state in the Root Microservice
Handoff output	Next-stage queue message with blob paths and metadata	Provides the downstream service with everything needed to continue

✓ Typical output organization in Blob Storage

`{fileId}/inference/*` for pass-1 segmentation results

`{fileId}/inference_pass2/*` for refined pass-2 segmentation results

`{fileId}/images/*` for extracted image assets

`{fileId}/postproc/walls.json` for wall geometry and related metadata

Integration Boundaries

Upstream: Root Microservice

The Root Microservice acts as orchestrator. It sends the segmentation task through the segmentation queue and later receives a workflow activity signal indicating success or failure. This allows orchestration state to be tracked independently of the heavy inference execution.

Downstream: next-stage processing

After uploads complete, Segmentation publishes a message containing artifact paths and metadata for the next stage. At this point, Segmentation's responsibility ends; downstream services consume the generated PLY files, images, and geometry metadata.

- ✔ **Handoff contract:** the downstream consumer does not need to rerun segmentation. It receives storage references to already prepared segmentation artifacts, image outputs, and supporting metadata generated by this service.

Failure Handling and Operational Behavior

Scenario	Observed behavior
No walls detected	The refined second pass may be skipped, and the service can continue with pass-1 outputs when available
GPU memory pressure	The message is treated as retryable and can be re-queued according to the messaging retry policy
Blob download failure	The operation is retried with backoff because the issue is often transient
Non-retryable processing error	The service signals failure upstream so workflow orchestration can take corrective action

Operationally, this service is sensitive to input size, GPU availability, and point-cloud quality. Monitoring for queue lag, GPU utilization, memory consumption, and failure rates is important because the service sits at the most compute-intensive stage of the pipeline.

- ⚠ **Operational caution:** segmentation is both data-heavy and compute-heavy. Failures often surface from scan quality issues, large scene sizes, missing wall cues, or GPU resource exhaustion rather than simple application bugs.

Key Takeaways

- The Segmentation Microservice is the ML-heavy processing stage between orchestration and downstream artifact consumption.
- Its two-pass design improves segmentation quality by combining broad semantic understanding with wall-bounded refinement.
- It is queue-driven, GPU-backed, and optimized for large E57 point clouds using streaming inference patterns.
- Its outputs form the complete pre-post-processing handoff package: segmented PLY files, extracted images, wall geometry, and workflow status.